BELKORCHI
Mohamed

2SIO2
03/11/25

TP3 - Serveur Web et service HTTPS

1. Configuration SERV-WEB : 3
- Configuration du réseau 3
- Sources.list et mise à jour du système Debian 4
2. Installation, configuration des services 5
- Installation d'Apache 5
- Installation de MariaDB 6
- Installation de PHP 7
- Installation de phpMyAdmin 8
- Virtual Host 10
3. Publication du site en ligne 12
4. Installation du service DNS 13

1. Configuration SERV-WEB :

- Configuration du réseau

La commande nano /etc/network/interfaces permet de modifier la configuration IP du serveur, qu'on va configurer de la sorte :

```
  GNU nano 8.4
# This file describes the network interfaces available on your
# and how to activate them. For more information, see interface

source /etc/network/interfaces.d/*

# The loopback network interface
auto lo
iface lo inet loopback

# The primary network interface
allow-hotplug ens18
iface ens18 inet static
address 172.16.141.1
netmask 255.255.0.0
gateway 172.16.0.1
dns-nameserver 172.16.141.2
```

La commande ip a, elle, permet d'afficher la configuration IP

```
2: ens18: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default qlen 1000
    link/ether bc:24:11:db:40:fa brd ff:ff:ff:ff:ff:ff
    altname enp0s18
    altname enxbc2411db40fa
    inet 172.16.141.1/16 brd 172.16.255.255 scope global ens18
       valid_lft forever preferred_lft forever
    inet6 fe80::be24:11ff:fedb:40fa/64 scope link proto kernel_ll
       valid_lft forever preferred_lft forever
```

- Sources.list et mise à jour du système Debian

Le fichier sources.list indique à Debian où chercher les paquets pour les installer ou les mettre à jour.
C'est la liste des dépôts que apt utilise.

On doit donc modifier le fichier et mettre les liens les pluys récents en faisant nano /etc/apt/sources/list.

```
#deb cdrom:[Debian GNU/Linux 13.0.0 _Trixie_ - Official amd64 NETINST with firmware 20250809-11:20]/ trixie contrib main non-free-firmware

deb http://deb.debian.org/debian/ trixie main non-free-firmware
deb-src http://deb.debian.org/debian/ trixie main non-free-firmware

deb http://security.debian.org/debian-security trixie-security main non-free-firmware
deb-src http://security.debian.org/debian-security trixie-security main non-free-firmware

# trixie-updates, to get updates before a point release is made;
# see https://www.debian.org/doc/manuals/debian-reference/ch02.en.html#_updates_and_backports
deb http://deb.debian.org/debian/ trixie-updates main non-free-firmware
deb-src http://deb.debian.org/debian/ trixie-updates main non-free-firmware

# This system was installed using small removable media
# (e.g. netinst, live or single CD). The matching "deb cdrom"
# entries were disabled at the end of the installation process.
# For information about how to configure apt package sources,
# see the sources.list(5) manual.
```

Après avoir modifier le fichier, on fait la commande apt update && upgrade pour rafraîchir la liste des paquets et pour mettre à jour tous les paquets installés.

```
root@Debian:/# nano etc/apt/sources.list
root@Debian:/# apt update && apt upgrade -y
Atteint : 1 http://deb.debian.org/debian trixie InRelease
Atteint : 2 http://deb.debian.org/debian trixie-updates InRelease
Atteint : 3 http://security.debian.org/debian-security trixie-security InRelease
10 paquets peuvent être mis à jour. Exécutez « apt list --upgradable » pour les voi
Mis à jour :
  libnss-systemd  libpam-systemd  libsystemd-shared  libsystemd0  libudev1  linux-i

Installation de dépendances :
  linux-image-6.12.43+deb13-amd64

Paquets suggérés :
  firmware-linux-free  linux-doc-6.12  debian-kernel-handbook

Sommaire :
  Mise à niveau de : 10. Installation de : 1Supprimé : 0. Non mis à jour : 0
Taille du téléchargement : 115 MB
  Espace nécessaire : 166 MB / 28,9 GB disponible

Réception de : 1 http://deb.debian.org/debian trixie-updates/main amd64 libsystemd0
Réception de : 2 http://deb.debian.org/debian trixie-updates/main amd64 systemd-sys
```

2. Installation, configuration des services

- Installation d'Apache

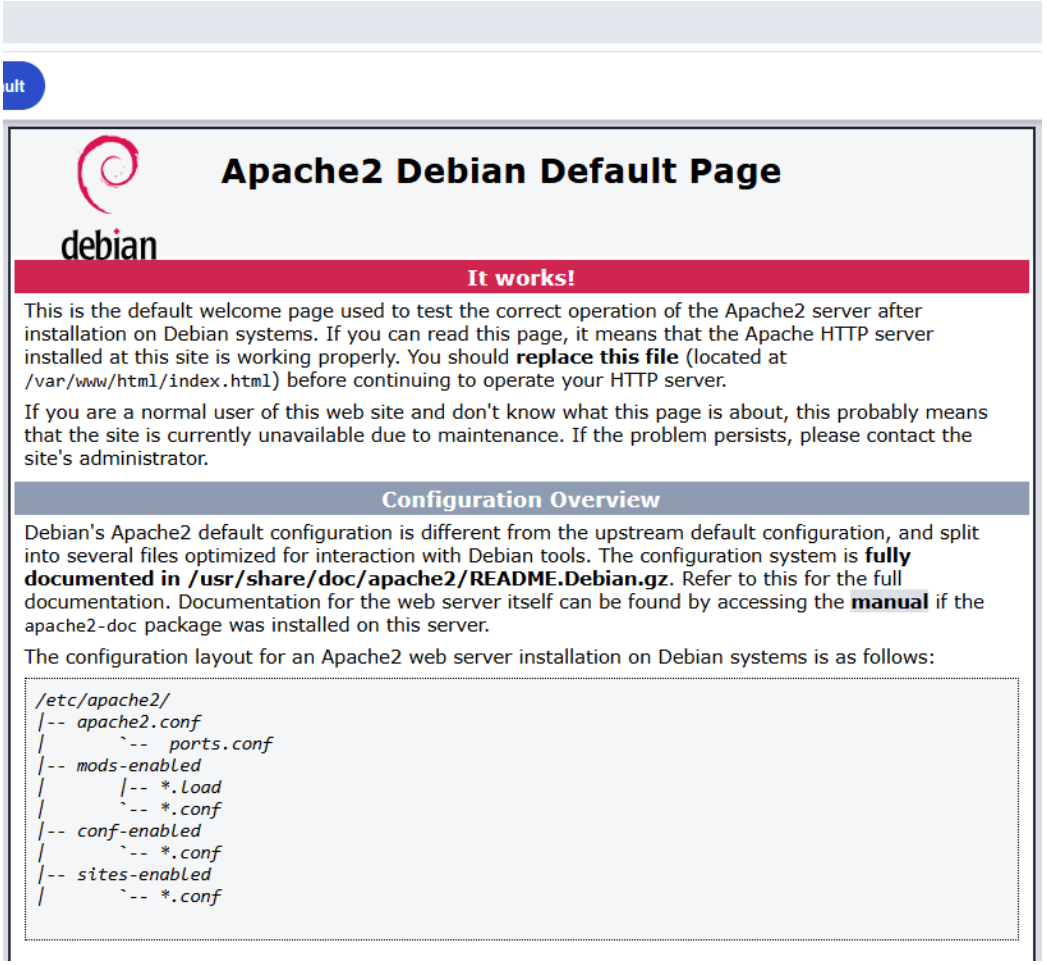
Apache est un serveur web qui sert à l'hébergement de sites et d'applications web sur Internet. On l'installe simplement avec la commande : apt-get install apache2.

```
root@Debian:/# apt install apache2 -y
Le paquet suivant a été installé automatiquement et n'est plus nécessaire :
  linux-image-6.12.38+deb13-amd64
Veuillez utiliser « apt autoremove » pour le supprimer.

Installation de :
  apache2

Installation de dépendances :
  apache2-bin   apache2-utils  libaprutil1-dbd-sqlite3  libaprutil1t64  libldap-common  liblua5.4-0   lib
  apache2-data  libapr1t64     libaprutil1-ldap         libcurl4t64     libldap2        libnghttp3-9  lib
```

Pour s'assurer que le service fonctionne bien, on se sert de la commande systemctl status apache2 , si on voit s'affiche en vert "active(running)" c'est que tout va bien.

```
root@Debian:/# systemctl enable apache2
Synchronizing state of apache2.service with SysV service script with /usr/lib/syste
Executing: /usr/lib/systemd/systemd-sysv-install enable apache2
^[[Aroot@Debian:/# systemctl start  apache2
root@Debian:/# systemctl status apache2
● apache2.service - The Apache HTTP Server
     Loaded: loaded (/usr/lib/systemd/system/apache2.service; enabled; preset: enab
     Active: active (running) since Mon 2025-09-15 16:18:17 CEST; 6min ago
 Invocation: 98c8101101544bce9281f7031cc18a6f
       Docs: https://httpd.apache.org/docs/2.4/
   Main PID: 7798 (apache2)
      Tasks: 55 (limit: 4640)
     Memory: 5.1M (peak: 5.4M)
```

Quand on tape l'adresse IP du serveur sur un navigateur, on est censé atterrir sur cette page, qui nous confirme que tout fonctionne.



Apache2 est donc installé et configuré.

- Installation de MariaDB

MariaDB est un système de gestion de bases de données qui stocke toutes les informations de ton site, comme les utilisateurs ou les articles. Pour l'installer on se sert de la commande : apt-get install mysql-server.Pour vérifier l'installation, comme pour Apache, on fait : systemctl status mariadb

```
root@Debian:/# systemctl status mariadb
● mariadb.service - MariaDB 11.8.3 database server
     Loaded: loaded (/usr/lib/systemd/system/mariadb.service; enabled; preset: enabled)
     Active: active (running) since Mon 2025-09-15 16:31:02 CEST; 57s ago
 Invocation: 0845ad2d50134250b3639d662aa4473a
       Docs: man:mariadbd(8)
             https://mariadb.com/kb/en/library/systemd/
```

- Installation de PHP

PHP est un langage côté serveur qui permet de créer des pages web dynamiques et interactives. Il s'installe avec les commandes :
apt-get install php
apt-get install libapache2-mod-php.

```
root@Debian:/# apt install php
Le paquet suivant a été installé automatiquement et n'est plus n
  linux-image-6.12.38+deb13-amd64
Veuillez utiliser « apt autoremove » pour le supprimer.

Installation de :
  php

Installation de dépendances :
  libapache2-mod-php8.4  php8.4

Paquets suggérés :
  php-pear
```

La commande php -v assure que php est bien installé et indique aussi sa version

```
root@Debian:/# php -v
PHP 8.4.11 (cli) (built: Aug  3 2025 07:32:21) (NTS)
Copyright (c) The PHP Group
Built by Debian
Zend Engine v4.4.11, Copyright (c) Zend Technologies
    with Zend OPcache v8.4.11, Copyright (c), by Zend Technologies
```

- Installation de phpMyAdmin

phpMyAdmin est une interface web qui permet de gérer facilement les bases de données MySQL/MariaDB sans passer par le terminal. On l'installe avec la commande : apt-get install phpmyadmin.

On peut accéder à l'interface graphique en entrant l'url suivant : 172.16.141.1/phpmyadmin.

Une fois installé, on s'y connecte pour y importer nos Base de Données.

Maintenant il vous faut retourner sur votre serveur Debian et créer des utilisateur MariaDB (1 compte admin et 2 comptes utilisateurs)

admin :

```
CREATE USER 'admin'@'localhost' IDENTIFIED BY 'Azerty31';

GRANT ALL PRIVILEGES ON *.* TO 'admin'@'localhost';

GRANT OPTION ON *.* TO 'admin'@'localhost';

FLUSH PRIVILEGES;
```

admin_backup :

```
CREATE USER 'admin_backup '@'localhost' IDENTIFIED BY 'Azerty31';

GRANT ALL PRIVILEGES ON *.* TO 'admin_backup '@'localhost';

GRANTT OPTION ON *.* TO 'admin_backup '@'localhost';

FLUSH PRIVILEGES;
```

site1_user :

```
CREATE USER 'site1_user'@'localhost' IDENTIFIED BY 'Azerty31';

GRANT SELECT, INSERT, UPDATE, DELETE ON site1.* TO

'site1_user'@'localhost';

FLUSH PRIVILEGES;
```

site2_user:

```
CREATE USER 'site2_user'@'localhost' IDENTIFIED BY 'Azerty31';

GRANT SELECT, INSERT, UPDATE, DELETE ON site2.* TO

'site2_user'@'localhost';

FLUSH PRIVILEGES;
```

Ce que vous êtes censé obtenir sur phpmyadmin:

Aperçu des comptes utilisateurs

	Nom d'utilisateur	Nom d'hôte	Mot de passe	Privilèges globaux	Groupe d'utilisateurs	« Grant »	Action		
☐	admin	localhost	Oui	ALL PRIVILEGES		Oui	Éditer les privilèges	Exporter	Verrouiller
☐	admin_backup	localhost	Oui	ALL PRIVILEGES		Oui	Éditer les privilèges	Exporter	Verrouiller
☐	mariadb.sys	localhost	Non	USAGE		Non	Éditer les privilèges	Exporter	Déverrouiller
☐	mysql	localhost	Oui	ALL PRIVILEGES		Oui	Éditer les privilèges	Exporter	Verrouiller
☐	phpmyadmin	localhost	Oui	USAGE		Non	Éditer les privilèges	Exporter	Verrouiller
☐	root	localhost	Oui	ALL PRIVILEGES		Oui	Éditer les privilèges	Exporter	Verrouiller
☐	site1_user	localhost	Oui	USAGE		Non	Éditer les privilèges	Exporter	Verrouiller
☐	site2_user	localhost	Oui	USAGE		Non	Éditer les privilèges	Exporter	Verrouiller

- Virtual Host

Un Virtual Host permet d'héberger plusieurs sites web sur le même serveur.

Il définit pour chaque site son nom de domaine, son dossier de fichiers et ses réglages spécifiques.

Pour en créer un sur Debian avec Apache, il suffit de créer un fichier dans /etc/apache2/sites-available/

On va donc créer le fichier site1.conf et modifier son contenu avec la commande : nano /etc/apache2/sites-available/site1.conf

```
<VirtualHost *:80>
    ServerName http://site1.local
    DocumentRoot /var/www/html/site1/
    DirectoryIndex cAccueil.php

    <Directory /var/www/html/site1>
        Options Indexes FollowSymLinks
        AllowOverride All
        Require all granted
    </Directory>

    ErrorLog ${APACHE_LOG_DIR}/site1_error.log
    CustomLog ${APACHE_LOG_DIR}/site1_access.log combined
</VirtualHost>
```

On va faire la même chose pour le site 2 :

nano /etc/apache2/sites-available/site2.conf

```
<VirtualHost *:80>
    ServerName http://site2.local
    DocumentRoot /var/www/html/site2/
    DirectoryIndex cAccueil.php

    <Directory /var/www/html/site2>
        Options Indexes FollowSymLinks
        AllowOverride All
        Require all granted
    </Directory>

    ErrorLog ${APACHE_LOG_DIR}/site2_error.log
    CustomLog ${APACHE_LOG_DIR}/site2_access.log combined
</VirtualHost>
```

Les sites seront accessibles en tapant site1.local et site2.local. Mais pour ça il faut configurer le serveur DNS

3. Publication du site en ligne

Nous allons maintenant rendre nos sites accessibles en ligne, il faudra tout d'abord lier nos sites à une base de donnée.

Pour ce faire, nous allons modifier ce fichier en faisant la commande

nano /var/www/html/site1/site/include/_bdGestionDonnees.lib.php

```php
<?php
/**
 * Regroupe les fonctions d'accès aux données.
 * @package default
 * @author Arthur Martin
 * @todo Fonctions retournant plusieurs lignes sont à réécrire.
 */

/**
 * Se connecte au serveur de données MySql.
 * Se connecte au serveur de données MySql à partir de valeurs
 * prédéfinies de connexion (hôte, compte utilisateur et mot de passe).
 * Retourne l'identifiant de connexion si succès obtenu, le booléen false
 * si problème de connexion.
 * @return resource identifiant de connexion
 */
function connecterServeurBD() {
    $hote = "localhost";
    $login = "site1_user";
    $mdp = "Azerty31";
    return mysqli_connect($hote, $login, $mdp);
}

/**
 * Sélectionne (rend active) la base de données.
 * Sélectionne (rend active) la BD prédéfinie gsb_frais sur la connexion
 * identifiée par $idCnx. Retourne true si succès, false sinon.
 * @param resource $idCnx identifiant de connexion
 * @return boolean succès ou échec de sélection BD
 */
function activerBD($idCnx) {
    $bd = "site1";
    $query = "SET CHARACTER SET utf8";
    // Modification du jeu de caractères de la connexion
    $res = mysqli_query($idCnx, $query);
    $ok = mysqli_select_db($idCnx, $bd);
    return $ok;
}
```

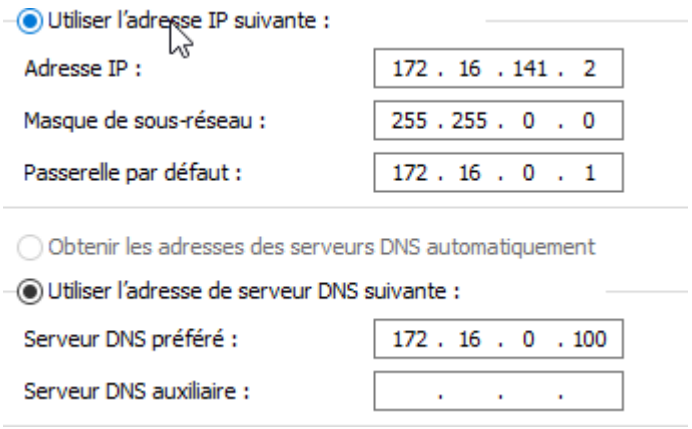
Faites la même chose pour le site2 en modifiant $login $mdp et $bd

Pour finir, fait la commande a2ensite site1.conf et a2ensite site2.conf pour activer les sites depuis le fichier .conf puis redémarrer le service en faisant systemctl restart apache2 pour qu'apache le prenne en compte.

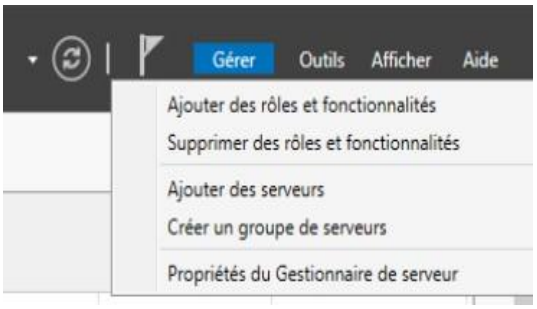
4. Installation du service DNS

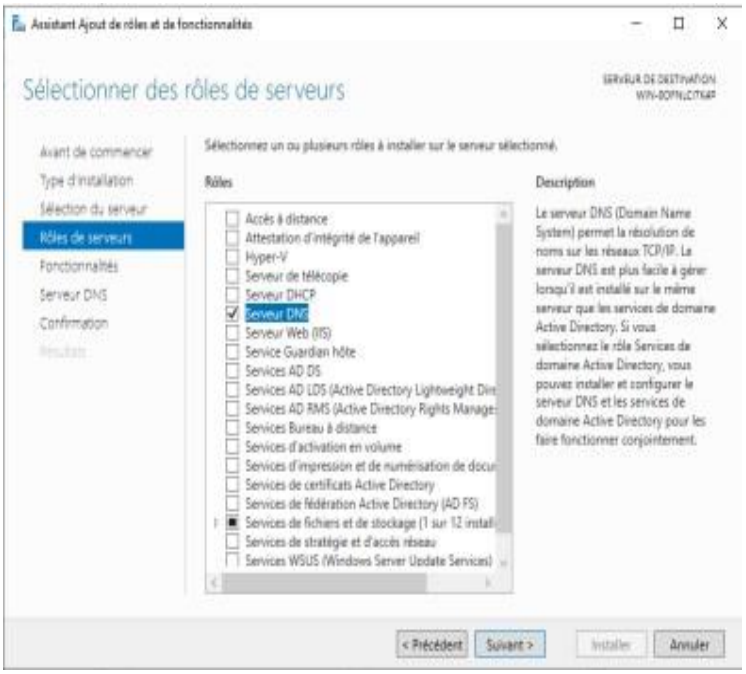
Le DNS (Domaine Name Service) va nous permettre de résoudre le nom de domaine en addresse IP de notre serveur web. Ainsi il sera possible d'accéder à nos deux sites respectif depuis http://site1.local et http://site2.local.

Notre DNS sera configuré de cette manière :

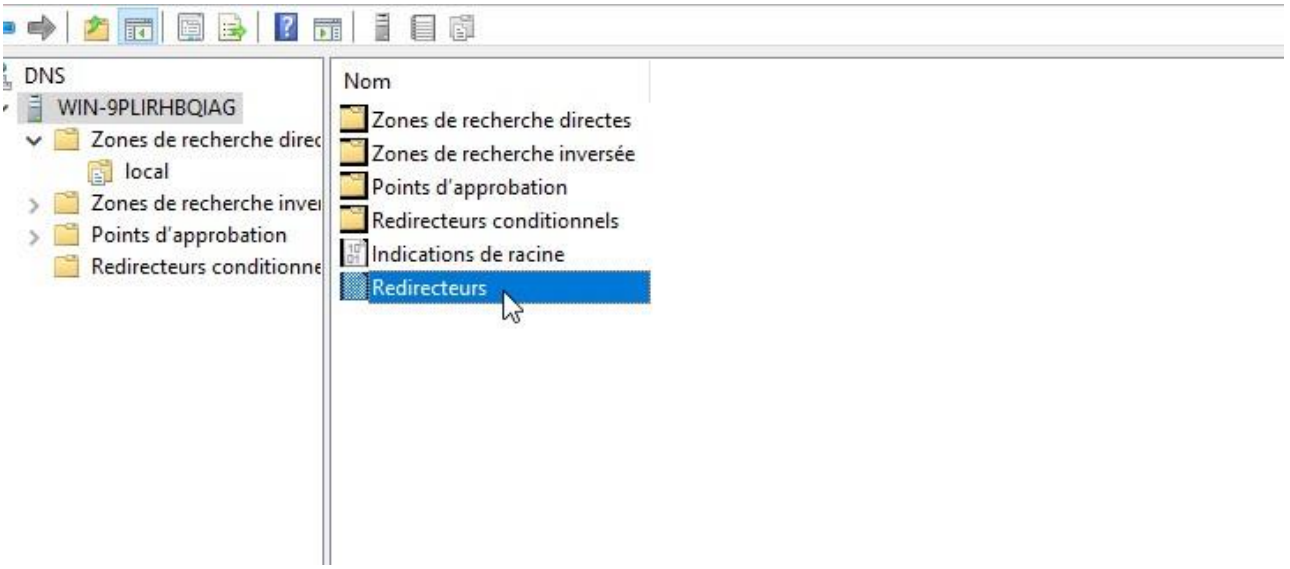


- <u>Installation du service DNS</u>

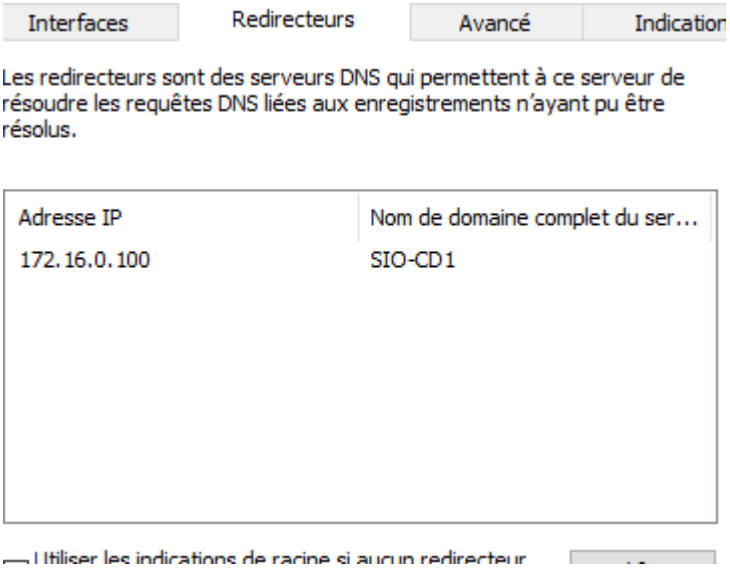




Faites suivant jusqu'à ce que le service s'installe. Tapez DNS dans la barre de recherche pour ouvrir la page de configuration du DNS

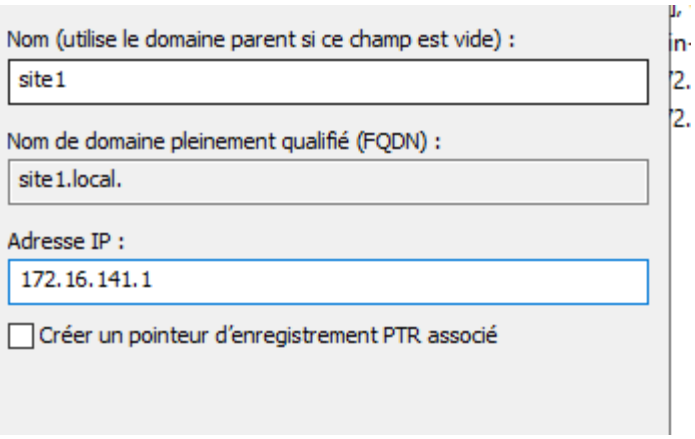


On va mettre le DNS du lycée pour notre serveur.

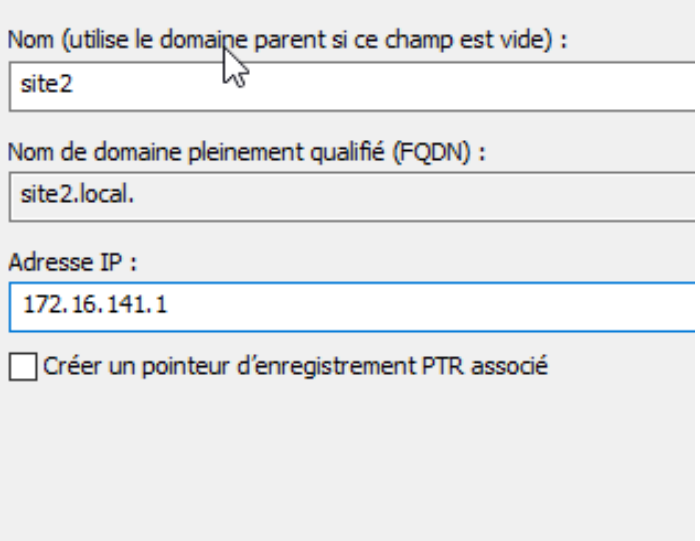


On va ensuite ajouter deux zones de recherches directes dans la zone local pour le site 1 et le site 2.

Pour le site 1, vous devrez avoir ceci



Et pour le site 2 ceci



On peut maintenant accéder à nos sites car on a configuré notre serveur DNS et les VirtualHost.

5. Certificat auto-signé

Avant de générer un certificat, il est nécessaire de créer une entité de confiance.

1. **Initialisation de l'arborescence** :

```
mkdir -p /root/demoCA/{certs,newcerts,crl,private}
touch /root/demoCA/index.txt
echo "01" > /root/demoCA/serial
chmod 700 /root/demoCA/private
```

2. **Génération de la clé privée de la CA**:

```
openssl genrsa -des3 -out /root/demoCA/private/cakey.pem 4096
```

3. **Génération du certificat racine auto-signé** :

```
openssl req -new -x509 -days 3650 -key /root/demoCA/private/cakey.pem -out
/root/demoCA/cacert.pem -sha256
```

Étape 2 : Création de la demande de certificat sur SRVWeb

1. **Génération de la clé privée du serveur** :

```
openssl genrsa -out /etc/ssl/private/debweb.key 2048
```

2. **Génération de la CSR**:
   Utilisation d'un fichier de configuration debweb_openssl.cnf pour définir le nom DNS et les informations de l'organisation :

```
openssl req -new -key /etc/ssl/private/debweb.key -out /root/debweb.csr -config
/root/debweb_openssl.cnf
```

Étape 3 : Signature du certificat par la CA

Le fichier debweb.csr a été transféré sur le serveur SRVcert via SCP, puis signé :

```
openssl x509 -req -in /root/debweb.csr -CA /root/demoCA/cacert.pem -CAkey
/root/demoCA/private/cakey.pem -CAcreateserial -out /root/demoCA/certs/debweb.crt -
days 825 -sha256
```

Étape 4 : Configuration d'Apache sur SRVWeb

Une fois le certificat signé (debweb.crt) récupéré, la configuration a été finalisée :

1. **Activation des modules SSL** :

```
a2enmod ssl
a2ensite default-ssl
```

2. **Modification du VirtualHost (port 443)** :

Pour le site 1:

```
<VirtualHost *:443>

   ServerName www.site1.local
   DocumentRoot /var/www/html/

   SSLEngine on
   SSLCertificateFile /etc/ssl/certs/debweb.crt
   SSLCertificateKeyFile /etc/ssl/private/debweb.key
   SSLCertificateChainFile /etc/ssl/certs/cacert_debcertificat.pem

</VirtualHost>
```

Pour le site 2 :

```
<VirtualHost *:443>

   ServerName www.site2.local
   DocumentRoot /var/www/html/

   SSLEngine on
   SSLCertificateFile /etc/ssl/certs/debweb.crt
   SSLCertificateKeyFile /etc/ssl/private/debweb.key
   SSLCertificateChainFile /etc/ssl/certs/cacert_debcertificat.pem

</VirtualHost>
```